FastAPI Path Operations for Teams - Other Models

Let's now update the **FastAPI** application to handle data for teams.

This is very similar to the things we have done for heroes, so we will go over it quickly here.

We will use the same models we used in previous examples, with the **relationship attributes**, etc.

Add Teams Models

Let's add the models for the teams.

It's the same process we did for heroes, with a base model, a **table model**, and some other **data models**.

We have a `TeamBase` **data model**, and from it, we inherit with a `Team` **table model**.

Then we also inherit from the `TeamBase` for the `TeamCreate` and `TeamPublic` **data models**.

And we also create a `TeamUpdate` **data model**.

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str


class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    name: str | None = None
    headquarters: str | None = None

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str


class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    name: str | None = None
    headquarters: str | None = None


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int


class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
```

```python
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}


@app.post("/teams/", response_model=TeamPublic)
def create_team(*, session: Session = Depends(get_session), team: TeamCreate):
    db_team = Team.model_validate(team)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.get("/teams/", response_model=list[TeamPublic])
def read_teams(
    *,
```

```
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    teams = session.exec(select(Team).offset(offset).limit(limit)).all()
    return teams


@app.get("/teams/{team_id}", response_model=TeamPublic)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team


@app.patch("/teams/{team_id}", response_model=TeamPublic)
def update_team(
    *,
    session: Session = Depends(get_session),
    team_id: int,
    team: TeamUpdate,
):
    db_team = session.get(Team, team_id)
    if not db_team:
        raise HTTPException(status_code=404, detail="Team not found")
    team_data = team.model_dump(exclude_unset=True)
    db_team.sqlmodel_update(team_data)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.delete("/teams/{team_id}")
def delete_team(*, session: Session = Depends(get_session), team_id: int):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    session.delete(team)
    session.commit()
    return {"ok": True}
```

We now also have **relationship attributes**. 🎉

Let's now update the `Hero` models too.

Update Hero Models

Python 3.10+

```
# Code above omitted 👆

class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int
```

```python
class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str


class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    name: str | None = None
    headquarters: str | None = None


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int


class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
```

```
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}


@app.post("/teams/", response_model=TeamPublic)
def create_team(*, session: Session = Depends(get_session), team: TeamCreate):
    db_team = Team.model_validate(team)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.get("/teams/", response_model=list[TeamPublic])
def read_teams(
    *,
```

```python
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    teams = session.exec(select(Team).offset(offset).limit(limit)).all()
    return teams


@app.get("/teams/{team_id}", response_model=TeamPublic)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team


@app.patch("/teams/{team_id}", response_model=TeamPublic)
def update_team(
    *,
    session: Session = Depends(get_session),
    team_id: int,
    team: TeamUpdate,
):
    db_team = session.get(Team, team_id)
    if not db_team:
        raise HTTPException(status_code=404, detail="Team not found")
    team_data = team.model_dump(exclude_unset=True)
    db_team.sqlmodel_update(team_data)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.delete("/teams/{team_id}")
def delete_team(*, session: Session = Depends(get_session), team_id: int):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    session.delete(team)
    session.commit()
    return {"ok": True}
```

We now have a `team_id` in the hero models.

Notice that we can declare the `team_id` in the `HeroBase` because it can be reused by all the models, in all the cases it's an optional integer.

And even though the `HeroBase` is *not* a **table model**, we can declare `team_id` in it with the `foreign key` parameter. It won't do anything in most of the models that inherit from `HeroBase`, but in the **table model** `Hero` it will be used to tell **SQLModel** that this is a **foreign key** to that table.

Relationship Attributes

Notice that the **relationship attributes**, the ones with `Relationship()`, are **only** in the **table models**, as those are the ones that are handled by **SQLModel** with SQLAlchemy and that can have the automatic fetching of data from the database when we access them.

Python 3.10+

```python
# Code above omitted 👆

class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str
```

```python
class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    name: str | None = None
    headquarters: str | None = None


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int


class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str


class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    name: str | None = None
    headquarters: str | None = None


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int


class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
```

```python
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}


@app.post("/teams/", response_model=TeamPublic)
def create_team(*, session: Session = Depends(get_session), team: TeamCreate):
    db_team = Team.model_validate(team)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.get("/teams/", response_model=list[TeamPublic])
def read_teams(
    *,
```

```
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    teams = session.exec(select(Team).offset(offset).limit(limit)).all()
    return teams


@app.get("/teams/{team_id}", response_model=TeamPublic)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team


@app.patch("/teams/{team_id}", response_model=TeamPublic)
def update_team(
    *,
    session: Session = Depends(get_session),
    team_id: int,
    team: TeamUpdate,
):
    db_team = session.get(Team, team_id)
    if not db_team:
        raise HTTPException(status_code=404, detail="Team not found")
    team_data = team.model_dump(exclude_unset=True)
    db_team.sqlmodel_update(team_data)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.delete("/teams/{team_id}")
def delete_team(*, session: Session = Depends(get_session), team_id: int):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    session.delete(team)
    session.commit()
    return {"ok": True}
```

Path Operations for Teams

Let's now add the **path operations** for teams.

These are equivalent and very similar to the **path operations** for the **heroes** we had before, so we don't have to go over the details for each one, let's check the code.

Python 3.10+

```
# Code above omitted 👆

@app.post("/teams/", response_model=TeamPublic)
def create_team(*, session: Session = Depends(get_session), team: TeamCreate):
    db_team = Team.model_validate(team)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.get("/teams/", response_model=list[TeamPublic])
def read_teams(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
```

```
):
    teams = session.exec(select(Team).offset(offset).limit(limit)).all()
    return teams


@app.get("/teams/{team_id}", response_model=TeamPublic)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team


@app.patch("/teams/{team_id}", response_model=TeamPublic)
def update_team(
    *,
    session: Session = Depends(get_session),
    team_id: int,
    team: TeamUpdate,
):
    db_team = session.get(Team, team_id)
    if not db_team:
        raise HTTPException(status_code=404, detail="Team not found")
    team_data = team.model_dump(exclude_unset=True)
    db_team.sqlmodel_update(team_data)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.delete("/teams/{team_id}")
def delete_team(*, session: Session = Depends(get_session), team_id: int):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    session.delete(team)
    session.commit()
    return {"ok": True}
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str


class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    name: str | None = None
    headquarters: str | None = None


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int


class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
```

```
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}


@app.post("/teams/", response_model=TeamPublic)
def create_team(*, session: Session = Depends(get_session), team: TeamCreate):
    db_team = Team.model_validate(team)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.get("/teams/", response_model=list[TeamPublic])
def read_teams(
    *,
```

```
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    teams = session.exec(select(Team).offset(offset).limit(limit)).all()
    return teams


@app.get("/teams/{team_id}", response_model=TeamPublic)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team


@app.patch("/teams/{team_id}", response_model=TeamPublic)
def update_team(
    *,
    session: Session = Depends(get_session),
    team_id: int,
    team: TeamUpdate,
):
    db_team = session.get(Team, team_id)
    if not db_team:
        raise HTTPException(status_code=404, detail="Team not found")
    team_data = team.model_dump(exclude_unset=True)
    db_team.sqlmodel_update(team_data)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.delete("/teams/{team_id}")
def delete_team(*, session: Session = Depends(get_session), team_id: int):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    session.delete(team)
    session.commit()
    return {"ok": True}
```

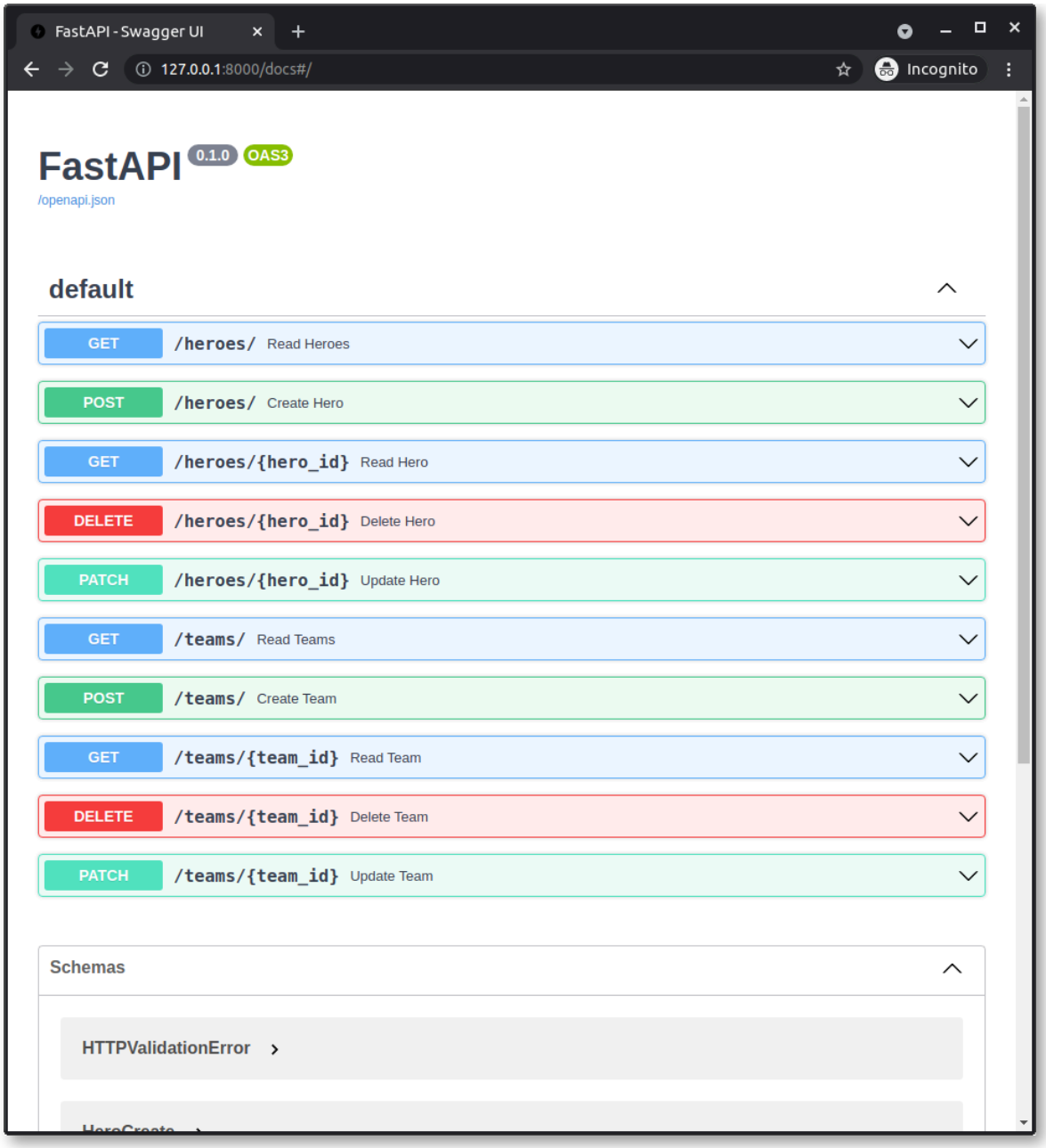
Using Relationships Attributes

Up to this point, we are actually not using the **relationship attributes**, but we could access them in our code.

In the next chapter, we will play more with them.

Check the Docs UI

Now we can check the automatic docs UI to see all the **path operations** for heroes and teams.



Recap

We can use the same patterns to add more models and API **path operations** to our **FastAPI** application. 🎉